

TIC TAC TOE

Akalya Sridharan
& Khushi Panchal

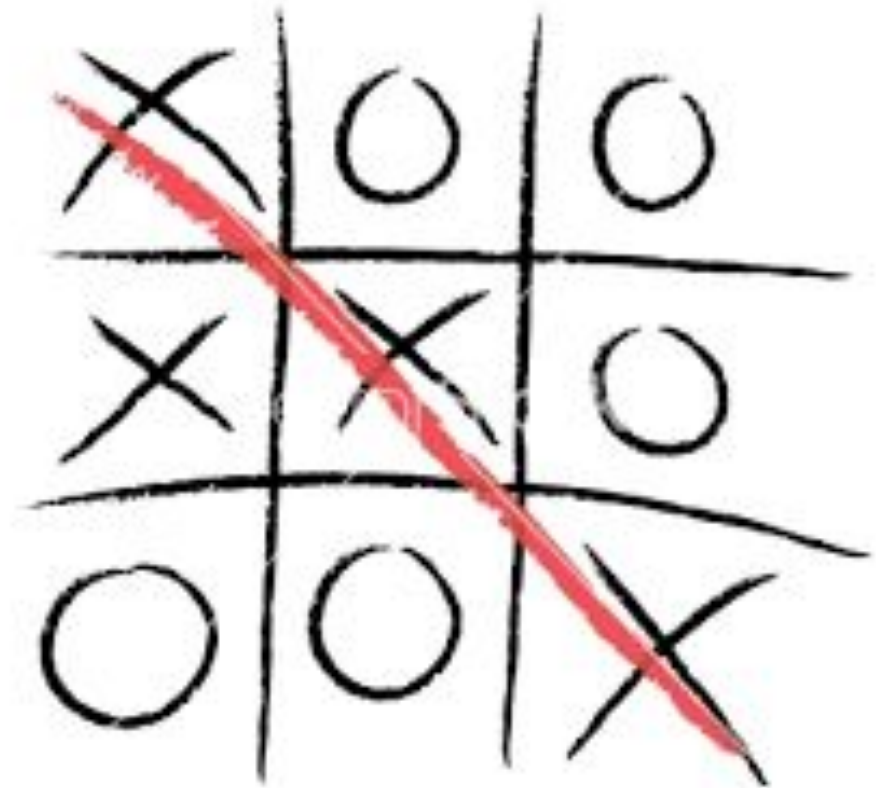


TABLE OF CONTENTS

1		Problem Motivation
2		The Minimax Algorithm
3		Alpha-Beta Pruning
4		Monte Carlo RL
5		Implementation Overview
6		Depth-Limited Minimax
7		Evaluation Methodology
8		Monte Carlo code review
9		Results & Insights

Problem Motivation

- Tic-Tac-Toe on 3×3 is predictable — often ending in draws.
- Scaling to $N \times N$ boards (4×4 , 5×5) exponentially increases complexity.
- New problems arise:
 - Larger state space.
 - More possibility trees to explore.
- Our Goal: Create an AI that plays optimally and responsively for any board size.
- Real-world inspiration: AI in complex games like Connect Four, Go, Chess.

The Minimax Algorithm

- Minimax recursively explores all possible game states.
- Max player (AI) tries to maximize the final score.
- Min player (Opponent) tries to minimize it.
- Utility values:
 - Win: +1
 - Loss: -1
 - Draw: 0
- Implementation Details (ttd.py):
 - `minimax(board, k, depth_limit)` orchestrates the decision.
 - `max_alpha_beta()` and `min_alpha_beta()` recursively traverse possibilities.

Alpha-Beta Pruning

- Problem: Pure Minimax is too slow for large boards.
- Solution: Alpha-Beta pruning cuts off irrelevant branches early.
- Alpha (α) = best already explored option for Maximizer.
- Beta (β) = best already explored option for Minimizer.
- When to prune: If $\beta \leq \alpha$, we stop exploring further down that branch.
- Result: Same move chosen, but far fewer nodes visited!

Monte Carlo Reinforcement Learning

- Problem : Algorithm like Minimax requires full search of the game tree.
- Solution: We use reinforcement Learning which solely acts through experience.
- No need to add pre programmed "good moves"
- Monte Carlo Reinforcement Learning is great for finite games like Tic-Tac-Toe.

MiniMax Implementation

- Frontend (game.py):
 - Built with pygame for dynamic visual interaction.
 - Background gradient and button hover effects for a polished UI.
- Backend (ttt.py):
 - Full game logic: player switching, move validation, win checking, AI moves.
- Key Features:
 - Adjustable NxN board size (user input).
 - Responsive tile resizing for different screen dimensions.
 - Replay feature with a "Play Again" button.

Depth-Limited Minimax

- Full tree exploration on large boards is infeasible
- Introduced a depth limit (e.g., 5) to cap recursion
- Trade-off: faster decisions vs. potentially non-optimal moves
- Prevents stack overflow and improves responsiveness

Demo Scenarios

- Human vs. AI: Play as X or O through interactive UI.
- AI vs. Random Moves: Benchmark AI intelligence.
- Dynamic Board Size:
 - Board resizes based on user input (e.g., 3×3, 5×5, 6×6).
- Play Again Feature:
 - Immediate board reset to replay without restarting the game.

Evaluation Methodology

- Metrics Recorded:
- Time per move (seconds).
- Win/loss/draw outcome (success rate vs human/random moves).
- Number of pruned nodes (optional for deeper analysis).
- Testing Setup:
 - Board Sizes: 3×3 , 4×4 , 5×5 .
 - Depth Limits: 3, 4, 5.
 - 20 games per configuration.

Results & Insights

- Alpha-Beta Pruning:
 - Reduced node evaluation by 40-60% on average.
- Depth 4:
 - 80% win rate against random opponent on 4×4.
- Responsiveness:
 - All moves under 1s on 3×3 and 4×4 boards.
- Scalability:
 - Works for any NxN with no additional code change — just user input.

Monte Carlo Implementation

- Frontend (tictactoe.py) :
 - Setup the board and reset.
 - Legal moves check as well as detects wins, draw for ongoing game
- Backend (mcts.py):
 - Uses ϵ - greedy policy to move.
 - Tracks the visited box during the game
- Features:
 - Learns completely by experience.

Training Agent

- Training Loop : resets the board, alternate moves between agent and opponent
 - After game, calculates the points:
 - +1 for winning.
 - 0 for draw.
 - -1 for loss.
- ϵ -greedy: Moves randomly with probability ϵ , or chooses the best move otherwise.

Results & Insights

- Average Move Time: $\sim 0.35s$
- Performance: MCTS consistently beats the opponents with high win rates.
- Optimal Play: At 1000 simulations, MCTS surpasses the performance of a perfect minimax agent.
- Trade-off: Higher simulation count improves quality but increases response time.

What We Learned

- Classic AI algorithms (Minimax + Alpha-Beta) remain powerful when optimized.
- Responsiveness is key for user engagement in real-time games.
- Future Improvements:
 - Add a heuristic evaluation for large unfinished boards.
 - Implement iterative deepening for smarter depth management.
- Link to GitHub repo and demo notebook for reproducibility
 - <https://github.com/akalyasri/Tic-Tac-Toe.git>

**THANK
YOU**

Khushi, Akalya